```
  nokogiri
  rspec
  solargraph
  standardrb

RUBY VERSION
   ruby 3.2.0p0

BUNDLED WITH
   2.3.3
```

This shows the actual version loaded for each gem, and in the case of debug, it shows the dependent gems (irb and reline) and the acceptable version range for each of them. It ends with a list of platforms that the bundle has been built for (this information is used for gems that include native code components and therefore need to be compiled locally by the operating system), the Ruby version, and the version of Bundler.

The lock file is there to be an exact snapshot of the versions of all the gems in use by the application. When bundle install is called again, if there is a Gemfile.lock, Bundler uses the information in the lock file to determine which version of each gem to install. The lock file, if it exists, is the most trusted source for which version of each gem should be installed.

If, after I install this bundle, there's a new version of RSpec released, my bundle will continue using the older version in the lock file until I explicitly update RSpec's version.

This is great because it means that any developer in the future who goes to work on this code base will automatically use Bundler to install the exact same set of gem versions as every other developer. (And of course, "any developer" doesn't just mean other people, it also means "you in six months.")

Bundle Exec and Bundle.require

Being able to load a specific set of gem versions is a good start, but you also need to be able to find those specific versions when you run your code. You might have multiple different applications that use different versions of RSpec. RubyGems will store all those different versions for you, but each individual application needs to be matched to the version it expects given its lock file. How can we ensure that when we require a gem we get the expected version.

Bundler provides a command called bundle exec that manages this bookkeeping for us. You use it by prepending bundle exec before whatever command you want to run, as in bundle exec rspec spec/my_spec.rb.

The bundle exec command does some manipulation of your environment. Basically, it modifies RubyGems itself to limit RubyGems to loading the gems in the bundle, and it also manages the command-line environment so that normal command-line things work without surprise. Calling rspec, for example, which is an executable script created by a gem requires some background environment management to make sure the correct version of RSpec is called.

Alternately, inside your code, before you use any gems, you can require bundler/setup, which will do the same RubyGems manipulation to manage references to gems, and will autoload the gems so you don't have to require them individually. Many frameworks, such as Ruby on Rails, do the Bundler setup as part of their regular boot process. That said, if you bundle exec code that also has a require bundler/setup that's fine, the management work will only be done once.